💡 WHY DYNAMIC PROGRAMMING?

Pure recursion re-computes identical subproblems exponentially ($O(2^N)$). Dynamic Programming (DP) **caches subproblem results** in a table, reducing runtime from exponential $O(2^N)$ to polynomial **O(N) or O(N²)**!

💡 THE 2 DP CONDITIONS

1. **Overlapping Subproblems:** The exact same state is computed multiple times in the recursion tree.
2. **Optimal Substructure:** Optimal solution of size N can be built from optimal solutions of smaller subproblems!

1 5-STEP FAANG DP WORKFLOW

1. **Define State:** What does `dp[i]` or `dp[i][j]` represent?
2. **Recurrence Relation:** `dp[i] = fn(dp[i-1], dp[i-2], ...)`
3. **Base Cases:** `dp[0] = 1`, `dp[1] = 1`
4. **Computation Order:** Bottom-up loop or Top-down memoization
5. **Space Optimization:** Can we compress 1D/2D table?

2 TOP-DOWN VS BOTTOM-UP

Fibonacci Example ($N=5$):

```
Top-Down (Memoization):
fib(5) -> fib(4) -> fib(3) -> Memo look up!

Bottom-Up (Tabulation):
dp[0]=0, dp[1]=1
dp[2]=1, dp[3]=2, dp[4]=3, dp[5]=5!
```

📚 PREREQUISITES & COMPLEXITY REASONING

DP Category	Time Complexity	Compressed Space
1D Linear (Climbing Stairs)	**O(N)**	**O(1) space!**
Unbounded Knapsack (Coin Change)	**O(N · Target)**	**O(Target) space**
0/1 Knapsack (Subset Sum)	**O(N · Target)**	**O(Target) reverse loop**
2D Grid / Strings (LCS / Edit)	**O(N · M)**	**O(M) 2-row rolling**

⚡ CLUES THAT SCREAM DP

1. *"Max / Min ways to reach target"* $\rightarrow$ 1D / Knapsack DP
2. *"Longest common subsequence / Edit distance"* $\rightarrow$ 2D String DP
3. *"Can partition array into 2 equal sum subsets"* $\rightarrow$ 0/1 Knapsack DP

1 TOP-DOWN MEMOIZATION TEMPLATE

```
public int solveTopDown(int n) {
  Integer[] memo = new Integer[n + 1];
  return helper(n, memo);
}
private int helper(int n, Integer[] memo) {
  if (n <= 1) return n;
  if (memo[n] != null) return memo[n]; // Memoization Cache Hi
t!
  memo[n] = helper(n - 1, memo) + helper(n - 2, memo);
  return memo[n];
}
```

2 BOTTOM-UP O(1) SPACE COMPRESSION

```
// Fibonacci / House Robber O(1) Space Compression:
public int rob(int[] nums) {
  int prev2 = 0, prev1 = 0;
  for (int num : nums) {
    int curr = Math.max(prev1, prev2 + num);
    prev2 = prev1;
    prev1 = curr;
  }
  return prev1;
}
```

PATTERN 1 CLIMBING STAIRS (1D Fibonacci Transition)

e.g. Climbing Stairs (LC 70), Min Cost Climbing Stairs (LC 746)

RECURRENCE FORMULA

`dp[i] = dp[i-1] + dp[i-2]`. Space optimized to $O(1)$ variables `prev1` and `prev2`.

TEMPLATE — LC 70

```
public int climbStairs(int n) {
  if (n <= 2) return n;
  int prev2 = 1, prev1 = 2;
  for (int i = 3; i <= n; i++) {
    int curr = prev1 + prev2;
    prev2 = prev1;
    prev1 = curr;
  }
  return prev1;
}
```

PATTERN 2 HOUSE ROBBER I & II (Non-Adjacent Selection)

e.g. House Robber I (LC 198), House Robber II (Circular LC 213)

CIRCULAR ROBBER TRICK

For circular houses, max robbery $= \max(\text{rob}(0..N-2), \text{rob}(1..N-1))$!

TEMPLATE — LC 213

```
public int rob(int[] nums) {
  if (nums.length == 1) return nums[0];
  return Math.max(robRange(nums, 0, nums.length - 2), robRange(nums, 1, nums.l
ength - 1));
}
private int robRange(int[] nums, int start, int end) {
  int prev2 = 0, prev1 = 0;
  for (int i = start; i <= end; i++) {
    int curr = Math.max(prev1, prev2 + nums[i]);
    prev2 = prev1; prev1 = curr;
  }
  return prev1;
}
```

PATTERN 3 LONGEST PALINDROMIC SUBSTRING (Expand Around Center)

e.g. Longest Palindromic Substring (LC 5), Palindromic Substrings (LC 647)

CENTER EXPANSION STRATEGY

Expand around 2N-1 centers (odd center `i, i` and even center `i, i+1`) in $O(N^2)$ time and $O(1)$ space!

TEMPLATE — LC 5

```
public String longestPalindrome(String s) {
  int start = 0, maxLen = 0;
  for (int i = 0; i < s.length(); i++) {
    int len1 = expand(s, i, i);     // Odd center
    int len2 = expand(s, i, i + 1); // Even center
    int len = Math.max(len1, len2);
    if (len > maxLen) { maxLen = len; start = i - (len - 1) / 2; }
  }
  return s.substring(start, start + maxLen);
}
private int expand(String s, int l, int r) {
  while (l>=0 && r
```

PATTERN 4 DECODE WAYS (1-Digit & 2-Digit Partitioning)

e.g. Decode Ways (LC 91)

STATE TRANSITION

If single char $\in [1..9]$, `dp[i] += dp[i-1]`. If 2-char $\in [10..26]$, `dp[i] += dp[i-2]`.

TEMPLATE — LC 91

```
public int numDecodings(String s) {
  int n = s.length(); int[] dp = new int[n + 1];
  dp[0] = 1; dp[1] = s.charAt(0) == '0' ? 0 : 1;
  for (int i = 2; i <= n; i++) {
    int one = Integer.parseInt(s.substring(i - 1, i));
    int two = Integer.parseInt(s.substring(i - 2, i));
    if (one >= 1 && one <= 9) dp[i] += dp[i - 1];
    if (two >= 10 && two <= 26) dp[i] += dp[i - 2];
  }
  return dp[n];
}
```

PATTERN 5 COIN CHANGE I (Fewest Coins to Make Amount) e.g. Coin Change (LC 322)

UNBOUNDED FORWARD LOOP

Infinite reuse of coins! Loop `a = coin..amount` forward: `dp[a] = Math.min(dp[a], 1 + dp[a - coin])`.

TEMPLATE — LC 322

```
public int coinChange(int[] coins, int amount) {
  int[] dp = new int[amount + 1];
  Arrays.fill(dp, amount + 1);
  dp[0] = 0;
  for (int a = 1; a <= amount; a++) {
    for (int coin : coins) {
      if (a >= coin) dp[a] = Math.min(dp[a], 1 + dp[a - coin]);
    }
  }
  return dp[amount] > amount ? -1 : dp[amount];
}
```

PATTERN 6 COIN CHANGE II (Total Combination Ways) e.g. Coin Change II (LC 518)

OUTER LOOP = COINS (PREVENTS PERMUTATION DUPLICATES!)

Outer loop over `coins`, inner loop over `amount` ensures combinations (not permutations) are counted!

TEMPLATE — LC 518

```
public int change(int amount, int[] coins) {
  int[] dp = new int[amount + 1];
  dp[0] = 1;
  for (int coin : coins) {
    for (int a = coin; a <= amount; a++) {
      dp[a] += dp[a - coin];
    }
  }
  return dp[amount];
}
```

PATTERN 7 PARTITION EQUAL SUBSET SUM (0/1 Knapsack Reverse Loop)

e.g. Partition Equal Subset Sum (LC 416), Target Sum (LC 494)

REVERSE LOOP SINGLE USE TRICK

For single-use elements, inner loop MUST run backwards from `target` down to `num` to prevent using same element twice in 1D array!

TEMPLATE — LC 416

```
public boolean canPartition(int[] nums) {
  int sum = 0; for (int n : nums) sum += n;
  if (sum % 2 != 0) return false;
  int target = sum / 2;
  boolean[] dp = new boolean[target + 1];
  dp[0] = true;
  for (int num : nums) {
    for (int j = target; j >= num; j--) { // Reverse loop for 0/1!
      dp[j] = dp[j] || dp[j - num];
    }
  }
  return dp[target];
}
```

PATTERN 8 LONGEST INCREASING SUBSEQUENCE (Binary Search Tails Array)

e.g. Longest Increasing Subsequence (LC 300)

PATIENCE SORTING O(N LOG N)

Maintain array `tails[]` of smallest tail of all increasing subsequences of length $i+1$. Use binary search (`Arrays.binarySearch`) to replace or append!

TEMPLATE — LC 300

```
public int lengthOfLIS(int[] nums) {
  int[] tails = new int[nums.length];
  int len = 0;
  for (int num : nums) {
    int i = Arrays.binarySearch(tails, 0, len, num);
    if (i < 0) i = -(i + 1);
    tails[i] = num;
    if (i == len) len++;
  }
  return len;
}
```

PATTERN 9 LONGEST COMMON SUBSEQUENCE (2D Matrix Match) e.g. Longest Common Subsequence (LC 1143)

MATCHING VS NON-MATCHING TRANSITION

If `text1[i-1] == text2[j-1]`: `dp[i][j] = 1 + dp[i-1][j-1]`. Else `dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1])`.

TEMPLATE — LC 1143

```
public int longestCommonSubsequence(String text1, String text2) {
  int m = text1.length(), n = text2.length();
  int[][] dp = new int[m + 1][n + 1];
  for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
      if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
        dp[i][j] = 1 + dp[i - 1][j - 1];
      } else {
        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
      }
    }
  }
  return dp[m][n];
}
```

PATTERN 10 EDIT DISTANCE (Insert, Delete, Replace Operations) e.g. Edit Distance (LC 72)

3 EDIT TRANSITIONS

`dp[i][j] = 1 + Min(Insert dp[i][j-1], Delete dp[i-1][j], Replace dp[i-1][j-1])`.

TEMPLATE — LC 72

```
public int minDistance(String word1, String word2) {
  int m = word1.length(), n = word2.length();
  int[][] dp = new int[m + 1][n + 1];
  for (int i = 0; i <= m; i++) dp[i][0] = i;
  for (int j = 0; j <= n; j++) dp[0][j] = j;
  for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
      if (word1.charAt(i - 1) == word2.charAt(j - 1)) dp[i][j] = dp[i - 1][j -
1];
      else dp[i][j] = 1 + Math.min(dp[i - 1][j - 1], Math.min(dp[i - 1][j], dp
[i][j - 1]));
    }
  }
  return dp[m][n];
}
```

DYNAMIC PROGRAMMING — WHICH PATTERN TO USE?

Start: What is the DP problem type?

- 🪜 **1D Steps / Non-adjacent items** → **1D DP / O(1) Space** `prev1` and `prev2`
- 🪙 **Unbounded item reuse (Coin Change)** → **Unbounded Knapsack** Forwa a = coin..a
- 🎒 **Single-use items (Subset Sum)** → **0/1 Knapsack** Reverse loop `j = target..num`
- 📈 **Subsequence ordering (LIS)** → **Patience Sorting LIS** Binary search `tails[]` array
- 🔤 **2-String matching (LCS / Edit)** → **2D String DP** `dp[m+1][n+1]` grid

🎯 TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Fewest coins to amount"	Unbounded Knapsack	Forward loop `1 + dp[a - coin]`
"Partition equal sum"	0/1 Knapsack	Reverse loop `dp[j] || dp[j - num]`
"Longest increasing subsequence"	LIS Patience Sort	Binary search replacement in `tails`
"Edit distance", "Min operations"	2D String DP	`1 + Min(Insert, Delete, Replace)`

💼 INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why reverse loop in 0/1 Knapsack?	A forward loop re-uses the SAME item multiple times (unbounded). A reverse loop ensures each item is used AT MOST ONCE!
Why outer loop over coins in Coin Change II?	Outer coin loop counts UNIQUE COMBINATIONS. Outer amount loop counts PERMUTATIONS!

EASY & MEDIUM — 1D DP

LC 70 · Climbing Stairs — EASY

Pattern: 1D Fibonacci
Key Insight: `prev1 + prev2`. Space $O(1)$.

```
public int climbStairs(int n) {
  if (n <= 2) return n;
  int p2 = 1, p1 = 2;
  for (int i = 3; i <= n; i++) {
    int c = p1 + p2; p2 = p1; p1 = c;
  }
  return p1;
}
```

LC 198 · House Robber — MEDIUM

Pattern: Non-Adjacent DP
Key Insight: `Math.max(p1, p2 + num)`.

```
public int rob(int[] nums) {
  int p2 = 0, p1 = 0;
  for (int n : nums) {
    int c = Math.max(p1, p2 + n); p2 = p1;
p1 = c;
  }
  return p1;
}
```

LC 213 · House Robber II — MEDIUM

Pattern: Circular Array DP
Key Insight: $\max(\text{rob}(0..N-2), \text{rob}(1..N-1))$.

```
public int rob(int[] nums) {
  if (nums.length == 1) return nums[0];
  return Math.max(r(nums, 0, nums.length -
2), r(nums, 1, nums.length - 1));
}
private int r(int[] n, int s, int e) {
  int p2 = 0, p1 = 0;
  for (int i=s; i<=e; i++) { int c = Math.
max(p1, p2 + n[i]); p2 = p1; p1 = c; }
  return p1;
}
```

MEDIUM — Palindromes & Coin Change

LC 5 · Longest Palindromic Substring — MEDIUM

Pattern: Center Expansion
Key Insight: Expand odd `i,i` & even `i,i+1`.

```
public String longestPalindrome(String s) {
  int st = 0, mx = 0;
  for (int i = 0; i < s.length(); i++) {
    int len = Math.max(exp(s, i, i), exp(s,
i, i + 1));
    if (len > mx) { mx = len; st = i - (len
- 1) / 2; }
  }
  return s.substring(st, st + mx);
}
```

LC 322 · Coin Change — MEDIUM

Pattern: Unbounded Knapsack
Key Insight: Forward loop `1 + dp[a - coin]`.

```
public int coinChange(int[] coins, int amou
nt) {
  int[] dp = new int[amount + 1]; Arrays.fi
ll(dp, amount + 1); dp[0] = 0;
  for (int a = 1; a <= amount; a++)
    for (int c : coins) if (a >= c) dp[a] =
Math.min(dp[a], 1 + dp[a - c]);
  return dp[amount] > amount ? -1 : dp[amou
nt];
}
```

MEDIUM — Subsets, LIS & LCS

LC 416 · Partition Equal Subset Sum — MEDIUM

Pattern: 0/1 Knapsack Reverse
Key Insight: Target $= \text{sum} / 2$. Reverse loop `j = target..num`.

```
public boolean canPartition(int[] nums) {
  int sum = 0; for (int n : nums) sum += n;
  if (sum % 2 != 0) return false;
  int target = sum / 2; boolean[] dp = new bo
olean[target + 1]; dp[0] = true;
  for (int n : nums)
    for (int j = target; j >= n; j--) dp[j] =
dp[j] || dp[j - n];
  return dp[target];
}
```

LC 300 · Longest Increasing Subsequence — MEDIUM

Pattern: LIS Patience Sort
Key Insight: Binary search tails array $O(N \log N)$.

```
public int lengthOfLIS(int[] nums) {
  int[] tails = new int[nums.length]; int len
= 0;
  for (int n : nums) {
    int i = Arrays.binarySearch(tails, 0, le
n, n);
    if (i < 0) i = -(i + 1);
    tails[i] = n; if (i == len) len++;
  }
  return len;
}
```

HARD — Edit Distance & LCS

LC 72 · Edit Distance — HARD

Pattern: 2D String Edit DP
Key Insight: `1 + Min(Insert, Delete, Replace)`.

```
public int minDistance(String w1, String w2)
{
  int m = w1.length(), n = w2.length();
  int[][] dp = new int[m + 1][n + 1];
  for (int i = 0; i <= m; i++) dp[i][0] = i;
  for (int j = 0; j <= n; j++) dp[0][j] = j;
  for (int i = 1; i <= m; i++) {
    for (int j = 1; j <= n; j++) {
      if (w1.charAt(i - 1) == w2.charAt(j -
1)) dp[i][j] = dp[i - 1][j - 1];
      else dp[i][j] = 1 + Math.min(dp[i - 1]
[j - 1], Math.min(dp[i - 1][j], dp[i][j -
1]));
    }
  }
  return dp[m][n];
}
```

LC 1143 · Longest Common Subsequence — MEDIUM

Pattern: 2D String Match
Key Insight: Match $= 1 + \text{diag}$. Else $\max(\text{up}, \text{left})$.

```
public int longestCommonSubsequence(String t
1, String t2) {
  int m = t1.length(), n = t2.length();
  int[][] dp = new int[m + 1][n + 1];
  for (int i = 1; i <= m; i++)
    for (int j = 1; j <= n; j++)
      dp[i][j] = (t1.charAt(i - 1) == t2.char
At(j - 1)) ? 1 + dp[i - 1][j - 1] : Math.max
(dp[i - 1][j], dp[i][j - 1]);
  return dp[m][n];
}
```

COMPLETE DYNAMIC PROGRAMMING PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
70	Climbing Stairs	1D Fibonacci	O(N)	O(1)	E
198	House Robber	Non-Adjacent DP	O(N)	O(1)	M
213	House Robber II	Circular Array DP	O(N)	O(1)	M
5	Longest Palindromic Substring	Center Expansion	$O(N^2)$	O(1)	M
322	Coin Change	Unbounded Knapsack	O(N · Amount)	O(Amount)	M
518	Coin Change II	Unbounded Combinations	O(N · Amount)	O(Amount)	M
416	Partition Equal Subset Sum	0/1 Knapsack Reverse	O(N · Target)	O(Target)	M
300	Longest Increasing Subsequence	LIS Patience Sort	O(N log N)	O(N)	M
1143	Longest Common Subsequence	2D String Match	O(N · M)	O(N · M)	M
72	Edit Distance	2D String Edit DP	O(N · M)	O(N · M)	H

🔍 DRY RUN — Coin Change ([1, 2, 5], Amount=11)

Amount	dp[a] Calculation	Best Coin Used	dp[a] Value
0	Base case	-	0
1	$1 + dp[0] = 1$	Coin 1	1
2	$\min(1+dp[1], 1+dp[0]) = 1$	Coin 2	1
5	$\min(1+dp[4], 1+dp[0]) = 1$	Coin 5	1
11	$1 + dp[6] = 1 + 2 = 3$ (Coins 5+5+1)	Coin 5	3 ✅

📐 MATHEMATICAL PROOF — O(N LOG N) LIS PATIENCE SORT

Claim: `tails[i]` stores the smallest tail of all increasing subsequences of length $i+1$.

Proof: `tails[]` is strictly increasing! For each number `x`, binary search finds the first element `tails[i] >= x`. Replacing `tails[i]` with `x` maintains the invariant with a smaller tail, creating more capacity for future elements $\rightarrow$ **$O(N \log N)$ Time!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Write 1D Climbing Stairs in $O(1)$ space
- ☐ Code House Robber II circular range split
- ☐ Implement Coin Change I unbounded loop
- ☐ Code Coin Change II combination loop
- ☐ Write 0/1 Knapsack reverse loop
- ☐ Code LIS in $O(N \log N)$ with binary search
- ☐ Write Longest Common Subsequence (LC 1143)
- ☐ Code Edit Distance (LC 72) in 2D grid

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Fill Array with Safe Infinity

In min-cost DP (e.g. Coin Change), initialize `dp[]` with `amount + 1` (NOT `Integer.MAX_VALUE` to avoid overflow during `1 + dp[a - coin]`)!

Rule 2 — Reverse Inner Loop for 0/1 Knapsack

When elements can only be used ONCE, inner loop MUST run backwards (`for (int j = target; j >= num; j--)`)!